Lab Assignment #5
CS3860

For this last Haskell assignment, we will write some code to work with 2-3 Trees.

Start by defining a data Type TTTree that will describe a 2-3 Tree expression. Remember that your data constructors must account for the fact that a 2-3 Tree has two types of internal nodes : 2-node and 3-node.

```
data TTTree a = -- FINISH THIS OFF
                deriving(Show, Eq, Ord)
```

Write the following functions :

Inorder:: TTTree a -> [a] - this will return a list containing the elements of the 2-3 Tree. This list is built by performing an inorder traversal of the 2-3 Tree.

height::TTTree a -> Int – This will return the height of a 2-3 Tree. An empty Tree will have height -1. Note that 2-3 trees are height-balanced (every leaf is at the same depth)

balance::TTTree a -> Bool – this function will return True if the 2-3 Tree is height-balanced, False otherwise.

Ordered::TTTree a -> Bool – this function will test that the 2-3 Tree is properly ordered. Hint : a properly ordered 2-3 Tree will produce a sorted list when traversed using inorder traversal.

Finally, functorize your TTTree type and provide an expression that that shows fmap at work. You can try creating a new expression of type TTTree that has its integer replaced by True/False to indicate if a certain value is present in the 2-3 Tree.

Document and submit a single .hs file. It will have a main action like the one shown below. Try creating your own 2-3 Trees.

```
main :: IO ()
main = do
  let t = -- YOUR 2-3 TREE EXPRESSION GOES HERE – USE INTEGERS
  putStrLn $ show t
  putStrLn $ show (inorder t)
  putStrLn $ show (height t)
  putStrLn $ show (balance t)
  putStrLn $ show (Ordered t)
  putStrLn $ show   $ fmap ( -- FILL THIS IN -- ) t
```